Plan 1 — Authentication, Registration & Role-Based Access

Status: Locked 2026-04-24 **Companion docs:** `plan-2-retention.md`, `plan-3-integration-scope.md`, `plan-4-integration-tech.md`, `plan-5-llm-cost.md`, `PLAN.md`, `DESIGN.md`, `product-validation.md`, `skillnex-alignment-review.md`

Executive summary

Skillnex moves from a single-tenant local demo to a multi-tenant SaaS with four roles (Owner, Admin, Manager, Employee). Public signup creates a tenant and Owner. Owners and Admins invite Managers and Employees. Each role sees a strictly scoped slice of the system — Managers only their direct reports, Employees only their own review. Tenant data is isolated by row-level `tenant_id` enforced through a single chokepoint, with a documented migration path to schema-per-tenant if a regulated client demands it. Auth runs on `better-auth` with email + password and magic-link recovery; SSO and 2FA come post-pilot. Everything runs on one DigitalOcean Droplet with one SQLite database for the pilot, migrating to managed Postgres when scaling demands it.

Locked decisions

#	Decision	Rationale
1	Auth library: **better-auth**	Modern, Next.js-first, multi-tenant friendly, Argon2id built in
2	Signup model: **public signup at `/signup`**	Lower friction; spam mitigated with rate limiting
3	2FA: **post-pilot**	Mandatory for Owner role added in Phase 3
4	Region picker at signup: **explicit US/EU choice**	Disclosed in privacy notice; default US
5	Employee accounts: **auto-created `status='invited'` on upload**	HR triggers actual invite emails when ready
6	User deletion: **soft-delete with 30-day recovery**; right-to-deletion bypasses grace	Aligns with privacy guide
7	Audit log retention: **tenant lifetime + 1 year**	Legal-defense window

Roles & permission matrix

Four roles, listed strongest-to-weakest privilege.

Capability	Owner	Admin	Manager	Employee
Billing	■	■	■	■
Delete tenant	■	■	■	■
Transfer ownership	■	■	■	■
Sign DPA	■	■	■	■
Configure integrations	■	■	■	■
Upload employee data	■	■	■	■
Invite users (any role)	■	■	■	■
See all employees	■	■	■	■
See own direct reports	■	■	■	■
Generate per-competency narratives for own reports	■	■	■	■
See own review (Part A + Part B)	■	■	■	■
Edit narrative drafts	■	■	■	■
Mark review complete / lock	■	■	■	■
View audit log	■	■	■	■
Export own data	■	■	■	■
Right-to-deletion (own data)	■	■	■	■
Right-to-deletion (someone else's)	■	■	■	■

Out of MVP, captured for phase 2:

- Multi-manager (one report has dotted-line manager)
- Calibration moderator role (read-only across the org)
- API tokens / service accounts
- Custom roles

Multi-tenancy architecture

Choice: row-level `tenant_id` with strict middleware enforcement.

Approach	Pros	Cons	Used for
Row-level `tenant_id` (selected)	One DB, simple deploys, dev/prod parity, fast to build	Requires disciplined query pattern; risk of leaks if the discipline breaks	Pilot + first ~100 customers
Schema-per-tenant	Architectural isolation; auditor-friendly	More moving parts, harder dev ergonomics	Migration when a regulated client demands it

Enforcement is single-chokepoint. No SQL is written outside `lib/db.ts` helpers. The helpers always require a `tenant_id` argument. Lint rule blocks `import { getDb } from "@/lib/db"` outside of `lib/db.ts` itself — every other file uses `scopedQuery(tenantId)`.

``ts // lib/db.ts export function scopedQuery(tenantId: string) { // Returns a wrapper whose every prepared statement injects WHERE tenant_id = ? // and rejects writes that don't include tenant_id in their parameters. } ``

API routes start with `requireTenantUser(req)` which returns `{ user, tenant }` or rejects.

Data model

```sql -- Tenant: one company / customer CREATE TABLE tenants ( id TEXT PRIMARY KEY, -- 'tnt<*base32*>' *name TEXT NOT NULL, -- 'Ryan Law Firm' region TEXT NOT NULL, -- 'us' | 'eu' plan TEXT NOT NULL DEFAULT 'pilot', retention*days INTEGER NOT NULL DEFAULT 90, -- post-cancellation grace; cap 30 for region='eu' created*at TEXT NOT NULL, deleted*at TEXT -- soft delete; hard-deleted by retention sweep );

-- Users with auth CREATE TABLE users ( id TEXT PRIMARY KEY, -- 'usr<*base32*>' *tenant*id TEXT NOT NULL REFERENCES tenants(id), email TEXT NOT NULL, email*normalized TEXT NOT NULL, -- lowercase + trim, used for login lookup password*hash TEXT, -- nullable: SSO users have no password (post-pilot) role TEXT NOT NULL, -- 'owner' | 'admin' | 'manager' | 'employee' name TEXT, status TEXT NOT NULL DEFAULT 'active', -- 'active' | 'invited' | 'suspended' | 'deleted' employee*key TEXT, -- if this user IS a reviewed employee, link email*verified*at TEXT, created*at TEXT NOT NULL, last*login*at TEXT, soft*deleted*at TEXT, -- 30-day grace window before hard delete UNIQUE(tenant*id, email*normalized) );

-- Sessions (server-side, revocable) CREATE TABLE sessions ( id TEXT PRIMARY KEY, -- random 32 bytes, this is the cookie value user*id TEXT NOT NULL REFERENCES users(id), tenant*id TEXT NOT NULL, expires*at TEXT NOT NULL, ip*address TEXT, user*agent TEXT, created*at TEXT NOT NULL );

-- Manager → list of employee*keys they can see CREATE TABLE manager*assignments ( manager*user*id TEXT NOT NULL REFERENCES users(id), employee*key TEXT NOT NULL, tenant*id TEXT NOT NULL, assigned*at TEXT NOT NULL, PRIMARY KEY (manager*user*id, employee*key) );

-- Invitations CREATE TABLE invitations ( token TEXT PRIMARY KEY, -- random URL-safe tenant*id TEXT NOT NULL, email TEXT NOT NULL, role TEXT NOT NULL, invited*by*user*id TEXT NOT NULL REFERENCES users(id), expires*at TEXT NOT NULL, -- 7 days accepted*at TEXT, created_at TEXT NOT NULL );

-- Append-only audit log CREATE TABLE audit*log ( id INTEGER PRIMARY KEY AUTOINCREMENT, tenant*id TEXT NOT NULL, user*id TEXT, -- null for system events action TEXT NOT NULL, -- 'login', 'view*employee',
```

'generate*narrative', 'export*data', etc. target*type TEXT, -- 'employee'* | *'tenant'* | *'user'* | *'narrative'* | *'integration*grant' target*id TEXT, ip*address TEXT, user_agent TEXT, details TEXT, -- JSON for context ts TEXT NOT NULL -- ISO 8601, UTC); -- No UPDATE or DELETE policy. Append-only.

-- Existing employees table extended ALTER TABLE employees ADD COLUMN tenant*id TEXT NOT NULL DEFAULT 'tnt*demo'; ALTER TABLE employees ADD COLUMN excluded*from*review INTEGER DEFAULT 0; -- partner-data exclusion per privacy guide ALTER TABLE employees ADD COLUMN integration*opt*out INTEGER DEFAULT 0; -- per Plan 3 CREATE INDEX idx*employees*tenant ON employees(tenant_id); ```

Auth flows

Sign-up (public)

- Visitor → `/signup`
- Form: company name, your name, work email, password, region (US/EU)
- POST creates `tenant` row + `user` row with `role='owner'` + sends email verification
- Click verification link → tenant becomes `active`, user signed in
- Onboarding wizard prompts upload OR integration connect

Spam mitigation: rate-limit signup IPs to 5 attempts/hour, email verification required before any data can be uploaded.

Invite (Owner/Admin invites teammates)

- `/app/settings/team` → "Invite teammate" form: email + role
- POST creates `invitation` row + sends email with `accept-invite/[token]` link
- Invitee clicks → enters name + password → user row created with chosen role + tenant scoped to inviter's tenant
- If `role='employee'`, system tries to match email to an existing `employees` row; if matched, links `users.employee_key`

Login

- `/login` → email + password → POST verifies, creates session row, sets HTTP-only cookie
- Audit-log: `action='login'`, IP + UA recorded
- Forgot password: `/forgot-password` → magic link emailed → link sets new password

Session security

- Cookie: HTTP-only, Secure, SameSite=Lax, 30-day expiry
- Server-side session table allows instant revocation
- "Sign out all sessions" button in `/app/settings/security`
- Session rotated on privilege change (role upgrade/downgrade)

Routes

Route	Public/Role	Purpose
`/`	Public	Marketing landing
`/signup`	Public	Create tenant + Owner
`/login`	Public	Sign in
`/forgot-password`	Public	Magic link
`/accept-invite/[token]`	Public (with valid token)	Complete account from invite
`/app/upload`	Owner, Admin	Ingest screen
`/app/dashboard`	Owner, Admin	Team overview
`/app/people`	Owner, Admin	All employees
`/app/people`	Manager	Filtered to direct reports
`/app/people/[key]/review`	Owner, Admin, that report's Manager	Per-competency review (built in alignment-memo Phase 2)
`/app/my-review`	Employee	Own review, read-only
`/app/calibration`	Owner, Admin	Scatter
`/app/integrations`	Owner, Admin	Configure connectors
`/app/settings/team`	Owner, Admin	Invite, change roles, suspend users
`/app/settings/profile`	Any logged-in	Name, email, password
`/app/settings/security`	Any	Session list, "sign out everywhere" (2FA placeholder)
`/app/settings/data`	Owner, Admin	DPA, data export, account delete (Plan 2)
`/app/settings/audit-log`	Owner, Admin	Filterable log per Plan 2 transparency requirement
`/app/settings/billing`	Owner	Plan, payment (post-pilot)

API routes use `requireRole(req, [...allowed])` middleware. Rejection writes a `permission_denied` audit-log event with caller details.

Email delivery

Resend for transactional email. ~$0 free tier covers pilot; $20/mo at growth.

Templates needed:

- Signup verification
- Magic link (password reset)
- Invitation

- Breach notification (Plan 2)
- Sub-processor change notice (Plan 3)
- Right-to-deletion confirmation (Plan 2)

Tenant isolation enforcement

Two pieces of code, never bypassed:

```ts // lib/auth/middleware.ts export async function requireTenantUser(req: Request): Promise<{ user: User; tenant: Tenant; }> { const session = await getSessionFromCookie(req); if (!session) throw new HttpError(401, "Unauthenticated"); const user = await getUserById(session.user*id); const tenant = await getTenantById(user.tenant*id); if (!tenant || tenant.deleted_at) throw new HttpError(401, "Tenant inactive"); return { user, tenant }; }

export function requireRole(allowed: Role[]) { return async (req: Request) => { const ctx = await requireTenantUser(req); if (!allowed.includes(ctx.user.role)) { await auditLog({ tenant*id: ctx.tenant.id, user*id: ctx.user.id, action: "permission_denied", details: { required: allowed, actual: ctx.user.role, route: req.url }, }); throw new HttpError(403, "Forbidden"); } return ctx; }; }

// lib/db.ts export function scopedDb(tenantId: string) { // Returns a wrapper whose .prepare() statements are inspected to ensure // every SELECT/UPDATE/DELETE has a WHERE tenant*id = ? clause and every // INSERT writes tenant*id. Throws at startup if a query lacks scoping. } ```

Lint rule: `import getDb` is allowed only in `lib/db.ts`. Other files use `scopedDb()`. CI fails the PR if violated.

## Hosting & DB choice

**Pilot: one DigitalOcean Droplet, one SQLite file on a persistent volume.**

- Droplet: $12/mo (2 GB RAM)
- Volume: $1/GB/mo, 10GB starts at $1
- Daily volume snapshot: ~$0.05/GB/snapshot
- Total infra: ~$15/mo for the pilot

Migration path to DigitalOcean Managed Postgres (~$15/mo) when:

- We need horizontal scaling (rare for HR analytics workload)
- A customer's security questionnaire demands managed DB with point-in-time recovery
- Whichever comes first

`better-auth` supports both SQLite and Postgres backends. Migration is mechanical when needed.

## Build sequence (5 days)
```

Day	Work
1	Schema migration: `tenants`, `users`, `sessions`, `invitations`, `manager_assignments`, `audit_log`. Add `tenant_id`, `excluded_from_review`, `integration_opt_out` to existing `employees`. Backfill demo data into one `tnt_demo` tenant.
2	Install better-auth. Sign-up + login + invite flows + email via Resend. Owner can invite Admin/Manager/Employee.
3	`requireTenantUser` + `scopedDb` + `requireRole` middleware. Gate every existing route. Lint rule for direct `getDb` use.
4	`/app/settings/team`, `/profile`, `/security`, `/audit-log`, `/data`. Audit-log writes from every role-relevant action.
5	Manager-scoped `/people`. Employee-scoped `/my-review`. End-to-end test: Owner uploads, Manager sees only their reports, Employee sees only own review.

Tests that prove it works

Test	Verifies
`tests/auth/signup.test.ts`	New tenant + Owner created on signup
`tests/auth/invite.test.ts`	Invite token accepted, user created with correct role + tenant
`tests/auth/role-gate.test.ts`	Manager rejected from `/app/upload`, Employee rejected from `/app/people`
`tests/auth/session-revoke.test.ts`	"Sign out all sessions" invalidates other sessions immediately
`tests/auth/tenant-leak.test.ts`	Manager from tenant A querying employee from tenant B → 404 (not 403, doesn't reveal existence)
`tests/auth/audit-log.test.ts`	Login, logout, role-change all produce immutable log entries
`tests/auth/lint.test.ts`	Direct `getDb` import outside `lib/db.ts` → CI fails
End-to-end: signup → upload → invite Manager → Manager sees only their reports → Employee sees only own review	Full role-isolation flow

Open items deferred

- SSO (Google/Microsoft) — Phase 3
- 2FA mandatory for Owner — Phase 3
- API tokens / service accounts — Phase 4
- Custom roles — Phase 4
- Multi-manager (dotted-line) — Phase 3
- Active Directory / SCIM provisioning — when first enterprise customer asks